FVS — Arquitectura enterprise híbrida

Análisis de video con IA: qué vive en la nube, qué vive en cada planta y cómo se conectan

Autor: Dev · Octubre 2026 · Complemento de “Análisis de video con IA en local” y “Arquitectura de autoservicio con catálogo de modelos”

Preguntas que responde este documento: ¿dónde se entrena un modelo nuevo (por ejemplo, de calidad)? ¿Cómo llega a una planta que lo quiere usar? ¿Se descarga en versión “lite”? ¿El VLM debe ser local? ¿Qué componentes van en la nube y cuáles en el edge? ¿Qué es lo más caro en el edge? ¿Hace falta K3s? ¿Qué es el agente edge, cómo se instala y cómo se vincula con la nube? ¿Cómo se mantiene todo fácil y sencillo?

1. Resumen
2. Arquitectura híbrida
3. Configurar vs. ejecutar
4. Ciclo de vida de un modelo nuevo
5. El VLM en planta
6. Costos en el edge
7. Una cámara con varios modelos
8. Datos y mensajería
9. Cómo llega la configuración a la planta
10. El agente edge
11. Instalación y enrolamiento
12. Qué instala el agente
13. ¿Hace falta K3s?
14. Cómo mantenerlo fácil
15. Qué hay que construir

1. Resumen

La regla general es: **en la nube se entrena, se gobierna y se distribuye; en la planta se detecta, se describe y se alerta.** El video nunca sale de la planta. A la nube solo suben eventos, métricas, miniaturas, clips cortos de eventos y las muestras que alguien aprueba para entrenar.

> **Aclaración importante:** el *pool* de modelos no vive en la nube. En la nube está el **registro** (el almacén de versiones) y el **catálogo** (la vitrina con fichas y aprobación). El pool, es decir los modelos cargados en GPU procesando video, está en cada planta.

El puente entre ambos mundos es el **agente edge**: un programa pequeño en el servidor de cada planta que pide la configuración a la nube, descarga lo necesario, lo aplica y reporta su estado. Casi todo se **configura** en la nube, desde el portal FVS, pero casi todo se **ejecuta** en la planta.

2. Arquitectura híbrida: nube y planta

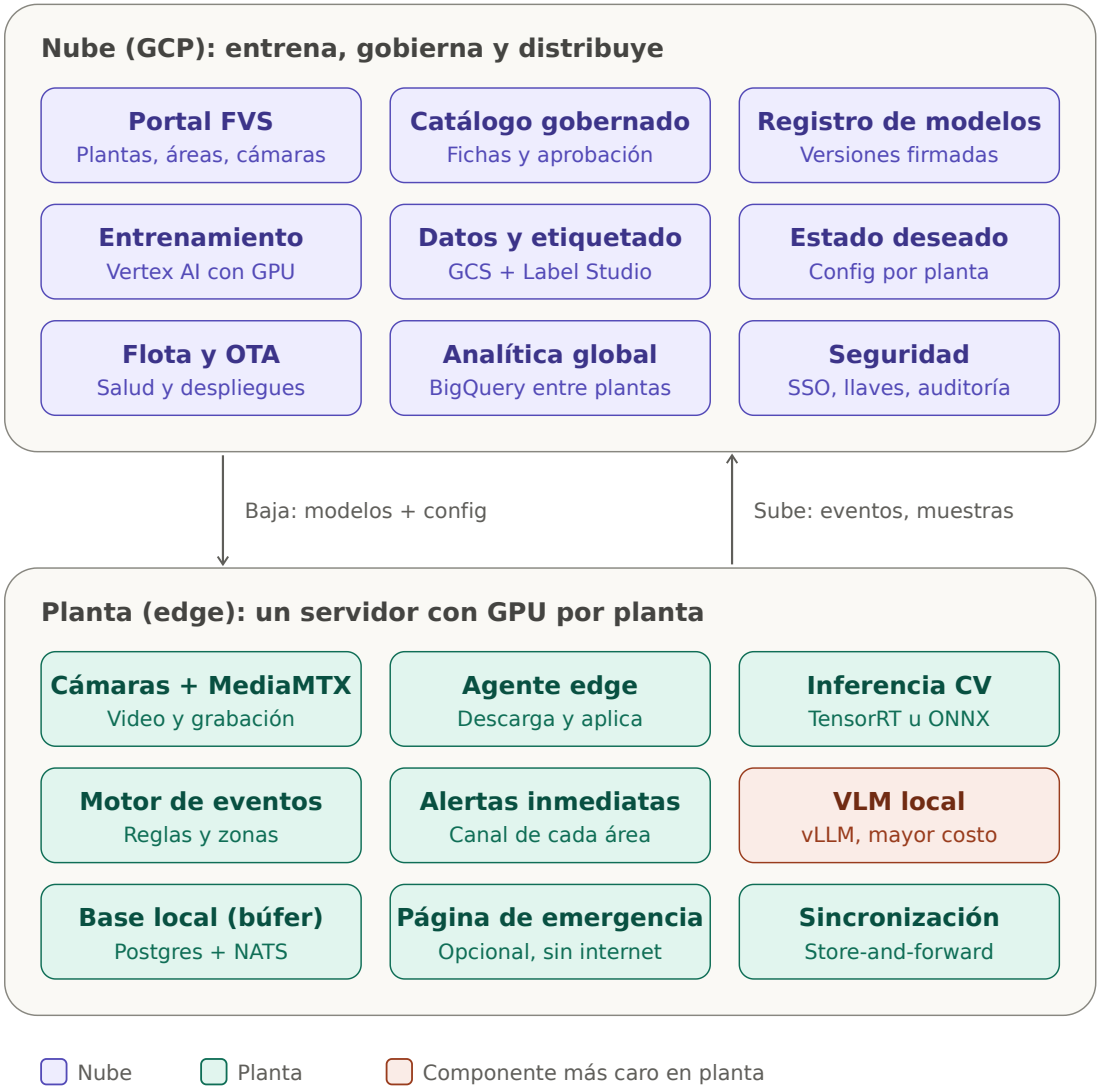


Figura 1. Arquitectura híbrida: qué vive en la nube y qué vive en cada planta.

Componentes en la nube (GCP)

Componente	Función
Portal FVS	Interfaz única para todas las plantas: alta de cámaras, elección de modelos, zonas, reglas, canales de aviso, política de VLM, consulta de eventos. Cada usuario ve solo su planta y sus áreas.
Catálogo gobernado	Capacidades disponibles con su ficha (eventos, reglas configurables, prompt, licencia, consumo de GPU) y proceso de aprobación.
Registro de modelos	Versiones de modelos en ONNX e imágenes de contenedor, firmadas. Cloud Storage y Artifact Registry.
Entrenamiento	Vertex AI con GPU por horas: pipeline reproducible y auditable.

Datos y etiquetado	Muestras aprobadas que suben las plantas, en Cloud Storage; etiquetado en Label Studio.
Estado deseado	Un documento de configuración versionado por planta, generado por el portal.
Flota y OTA	Salud de cada servidor edge y despliegues graduales de modelos y software.
Analítica global	Eventos de todas las plantas en AlloyDB (con pgvector) y BigQuery para reportes y búsqueda entre plantas.
Seguridad	SSO, certificados de los servidores edge, secretos (Secret Manager) y auditoría de cambios.

Componentes en la planta (edge)

Componente	Función
Cámaras + MediaMTX	Recibe los streams RTSP y graba. Las grabaciones completas se quedan en la planta.
Agente edge	Pide configuración, descarga modelos y servicios, los aplica y reporta salud.
Inferencia CV	Detectores y clasificadores optimizados con TensorRT (u ONNX Runtime sin GPU NVIDIA).
Motor de eventos	Convierte detecciones en eventos según reglas y zonas de cada cámara.
Alertas inmediatas	Envía las alertas al canal de cada área directamente desde la planta.
VLM local	Describe los eventos marcados para VLM. Es el componente más caro de la planta.
Búfer local	Guarda eventos mientras no hay internet (NATS JetStream y una base pequeña).
Sincronización	Envía eventos, clips cortos y métricas a la nube con store-and-forward.
Página de emergencia	Opcional: vista mínima de alertas recientes cuando no hay internet.

3. Configurar vs. ejecutar

La confusión más común es mezclar dónde se **configura** algo con dónde se **ejecuta**. Las reglas, zonas y alertas sí se configuran en la nube; lo que no conviene es ejecutarlas allá, porque cada frame tendría que viajar por internet antes de decidir si hay una alerta.

Componente	Se configura en	Se ejecuta en	Por qué se ejecuta ahí
Cámaras + MediaMTX	Portal (IT da de alta)	Planta	El video no debe salir de la planta
Inferencia CV	Portal (el área elige modelos)	Planta	Analiza muchos frames por segundo; subirlos sería carísimo
Reglas y zonas	Portal (el área dibuja y ajusta)	Planta	Necesita los resultados de la inferencia al instante

Alertas	Portal (canal y destinatarios)	Planta	Más rápidas y no dependen de que la nube esté disponible
VLM	Portal (qué eventos lo usan)	Planta	Recibe imágenes: privacidad y ancho de banda
Base de datos	—	Nube (principal) + planta (búfer)	Consolidación central sin perder eventos si se cae internet
Interfaz	—	Nube (portal FVS)	Una sola interfaz para todas las plantas
Sincronización	—	Planta	Es la que empuja los eventos hacia la nube

4. Ciclo de vida de un modelo nuevo (ejemplo: calidad)

¿Se entrena local o en la nube?

El prototipo puede hacerse local (laptop o GPU propia). El **modelo oficial** se entrena en la nube (Vertex AI con GPU) por tres razones:

- **Reproducible:** cualquiera puede volver a entrenarlo con el mismo pipeline, datos y versión de código.
- **Auditable:** queda registro de qué datos, qué métricas y quién lo aprobó.
- **Datos de varias plantas:** un modelo de calidad mejora con ejemplos de varias líneas, y eso solo se junta en un lugar central.

Flujo completo

1. **Muestras:** la planta sube solo frames o clips seleccionados (defectos, falsos positivos marcados por operadores), no video completo.
2. **Etiquetado** en Label Studio en la nube, con pre-etiquetado automático.
3. **Entrenamiento** en Vertex AI con GPU por horas: se paga solo mientras entrena.
4. **Evaluación** contra un set de prueba de cada planta donde se usará. Si no supera el umbral, no avanza.
5. **Registro:** se exporta a ONNX, se firma y se guarda como versión con su ficha.
6. **Aprobación:** un responsable lo publica en el catálogo.
7. **Activación:** un área lo elige en el portal; eso cambia el estado deseado de su planta.
8. **Descarga:** el agente edge detecta el cambio, descarga el modelo, verifica la firma y lo compila para su GPU.
9. **Despliegue gradual:** primero en una cámara (canary) y luego en todas. Si las métricas empeoran, regresa solo a la versión anterior.

¿Se descarga en versión “lite”?



Figura 2. Formatos del modelo: se entrena en la nube, se publica en ONNX y se compila en la planta.

- **Lo que se publica es ONNX**, un formato universal: el mismo archivo sirve para cualquier hardware. Se exporta una sola vez, al publicar.
- **La optimización final ocurre en la planta.** El agente compila el ONNX a TensorRT en el propio equipo, porque un motor TensorRT queda ligado al modelo exacto de GPU y a la versión de software.
- **Cuantización FP16 o INT8:** 2–4 veces más rápido con mínima pérdida de precisión. INT8 requiere imágenes de calibración, que pueden viajar en el paquete del modelo.
- Sin GPU NVIDIA (por ejemplo, la Mac del MVP) se usa el ONNX directo con ONNX Runtime, sin compilar.

Los detectores ya son ligeros de origen (aprox. 5–100 MB). El único pesado es el VLM.

5. El VLM en planta

Para la mayoría de las plantas, el VLM debe ser local por cuatro razones: privacidad (el video no sale), ancho de banda (mandar imágenes todo el día satura el enlace), latencia y costo predecible (sin pago por token). Sus pesos se descargan una vez en versión cuantizada (FP8 o 4-bit) y se actualizan rara vez.

> **Excepción útil:** una planta chica con pocos eventos puede no tener VLM local y mandar solo los frames del evento a un VLM en la nube (por ejemplo, Gemini en Vertex AI). Conviene que el diseño permita elegir por planta: VLM local o VLM en la nube.

6. Costos en el edge: ¿qué es lo más caro?

El VLM, por mucho. Valores aproximados; dependen del modelo y la GPU, así que deben medirse en el hardware real.

	Detector (YOLO, RT-DETR, clasificador)	VLM (ej. Qwen3-VL 8B)
Peso del modelo	~5–100 MB	~16 GB en FP16; ~5–9 GB cuantizado
Memoria de GPU	~0.5–2 GB	Pesos + caché de contexto e imágenes: varias veces más
Tiempo por análisis	Milisegundos por frame	~1–3 s por fragmento de 4–8 frames

Capacidad de una GPU	Decenas de cámaras a 5 fps	Decenas de fragmentos por minuto
De qué depende el costo	Cámaras × fps	Número de eventos enviados al VLM

Muchos detectores juntos cuestan menos que un VLM. El VLM ocupa la mayor parte de la memoria de la GPU y define qué tarjeta hay que comprar. Por eso la política de “qué eventos pasan por el VLM” es la principal palanca de costo.

Costos ocultos

- **Decodificar video:** en CPU satura el procesador con pocas cámaras; con el decodificador por hardware de la GPU (NVDEC) es casi gratis. Usar siempre el substream para la IA.
- **Almacenamiento de grabación:** una cámara 1080p a 4 Mbps genera unos 43 GB por día, más de 1 TB al mes. Con 20 cámaras y 30 días de retención son unos 26 TB. Se reduce con H.265, menor bitrate o retención larga solo de clips con eventos.

Perfiles de hardware por planta

Perfil	Hardware sugerido
Planta chica sin VLM local	Equipo compacto tipo NVIDIA Jetson Orin o GPU de entrada; solo detectores, VLM en la nube.
Planta típica con VLM local	Un servidor con una GPU de 24–48 GB (clase L4, RTX 4000/5000 Ada o similar).
Planta grande	Dos GPUs: una para detectores y otra dedicada al VLM, para que el VLM no frene la detección.

7. Una cámara con varios modelos

Ejemplo: una cámara configurada con seguridad, conteo de personas y cascos/chalecos.

No siempre son tres modelos. El conteo de personas no es un modelo aparte: es una regla sobre el detector de personas. Y el modelo de EPP puede detectar persona, casco y chaleco a la vez. Por eso el catálogo distingue entre **capacidades** (lo que el área elige: conteo, seguridad, EPP) y **modelos** (los archivos que corren en la GPU). Si dos capacidades usan el mismo modelo, el frame se analiza una sola vez y el resultado alimenta ambas reglas.

Sí, viven en el mismo servidor que el VLM y comparten la GPU. Ejemplo con una GPU de 24 GB:

Componente	Memoria aproximada
VLM cuantizado + caché de contexto	~10–14 GB
Detector de personas	~1 GB
Modelo de EPP	~1 GB
Decodificación de video	~1 GB

Margen libre	~7–11 GB

Cada modelo se carga **una sola vez para todas las cámaras**: si 15 cámaras usan EPP, hay un solo modelo de EPP en memoria procesando frames de las 15.

8. Datos y mensajería

Base de datos

La base principal va en la nube: **AlloyDB con pgvector** para eventos, descripciones y búsqueda semántica de todas las plantas, y **BigQuery** para reportes históricos. En la planta queda solo un **búfer**, por dos razones:

- Si se cae internet, la planta sigue detectando y los eventos se guardan hasta que vuelva la conexión.
- Las grabaciones completas viven en la planta; a la nube suben solo el clip corto y la miniatura de cada evento (pocos MB, a Cloud Storage), para que el portal los muestre sin depender de la conexión con la planta.

Interfaz

La interfaz de la planta es parte del portal FVS: una sola interfaz en la nube donde cada usuario ve su planta. Opcionalmente, una página mínima local para ver alertas cuando no haya internet.

¿Dónde entraría Kafka?

Kafka es una “tubería” de mensajes. En esta arquitectura hay dos tuberías y para ninguna es la mejor opción:

Tubería	Recomendado	Por qué
Dentro de la planta (detectores → eventos → VLM → búfer)	NATS JetStream (o Redis Streams)	Kafka es pesado para un solo servidor. NATS es ligero, persiste en disco y hace store-and-forward de forma natural.
De la planta a la nube	Google Pub/Sub	Administrado, sin operar brokers. De ahí los eventos van a AlloyDB y BigQuery.

La sincronización queda muy simple: un servicio en la planta que lee de NATS y publica en Pub/Sub, y solo borra un mensaje cuando la nube confirma que lo recibió. Kafka solo tendría sentido si el cliente ya lo usa como estándar corporativo; en ese caso se conecta desde la nube, no desde la planta.

9. Cómo llega la configuración a la planta

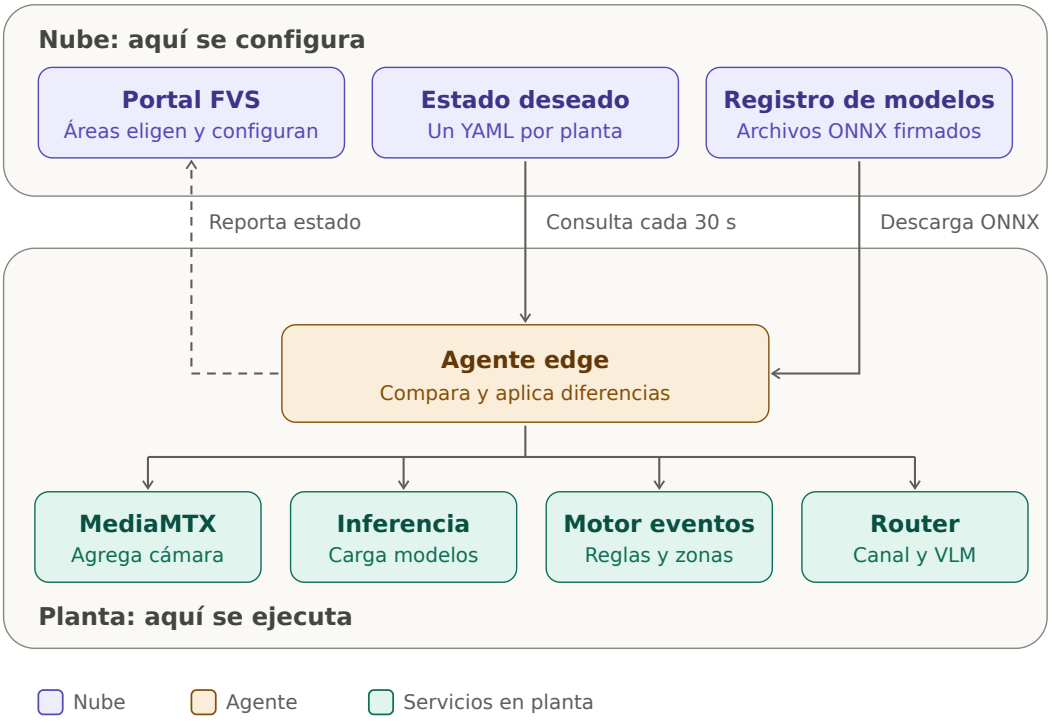


Figura 3. Cómo llega la configuración de la nube a la planta.

Ejemplo: el área de producción activa en el portal la cámara 12 con EPP y conteo, dibuja la zona “línea 1”, elige Teams de seguridad como canal y pide VLM solo para “persona sin casco”.

Paso 1. El portal genera el documento de configuración de la planta, con número de versión:

```yaml
planta: monterrey
version: 42
modelos:
  - id: detector-personas
    version: "2.1"
  - id: epp-cascos-chalecos
    version: "3.0"
camaras:
  - id: cam-12
    area: produccion
    rtsp: rtsp://10.0.5.12:554/Streams/Channels/102   # credenciales en un secreto aparte
    analisis:
      - modelo: epp-cascos-chalecos
        fps: 5
        zonas: [linea-1]
        reglas:
          - evento: persona_sin_casco
            persistencia_seg: 3
            notificar: teams-seguridad
            vlm: true
      - modelo: detector-personas
        fps: 2
        reglas:
          - evento: aforo_excedido
            umbral: 20
            notificar: telegram-produccion
            vlm: false
```

Las credenciales de las cámaras nunca van en este archivo: viajan cifradas por separado (Secret Manager).

Paso 2. El agente detecta la versión 42 (tenía la 41) y la descarga.

Paso 3. Aplica solo las diferencias:

- ¿Falta el modelo EPP v3.0? Lo descarga, verifica la firma, lo compila a TensorRT y lo carga en inferencia sin detener los demás.
- ¿Falta la cámara 12? La agrega a MediaMTX por su API.
- ¿Hay reglas y zonas nuevas? Las entrega al motor de eventos, que las recarga en caliente.
- ¿Hay canal y política de VLM nuevos? Los entrega al router.

Paso 4. Reporta “versión 42 aplicada” y el portal muestra la cámara en verde. Si algo falla, regresa a la versión 41 y el portal muestra el error.

Distribución de carga

Con una GPU no hay nada que distribuir: un solo servicio de inferencia atiende a todas las cámaras. La carga se controla en el portal **antes** de aplicar: el agente reporta la capacidad libre de la GPU y el portal estima el consumo de la nueva configuración (cámaras × fps × costo de cada modelo, más los eventos que irán al VLM). Si no cabe, avisa o pide aprobación. Con dos GPUs, el documento indica qué GPU usa cada cosa.

> **Si se cae la nube**, la planta sigue trabajando con la última configuración aplicada, porque el agente la guarda en disco. La nube solo hace falta para cambiar la configuración, no para operar.

10. El agente edge: el “robot” de la planta

El agente es el único componente de la planta que habla con la nube. Se encarga de descargar, instalar, configurar y reportar. **No analiza video**: solo administra los servicios que sí lo hacen. Si el agente se cae, la planta sigue detectando y alertando; solo deja de recibir cambios desde el portal mientras tanto.

Cómo se le llama: agente edge (edge agent), agente de nodo o agente de dispositivo. Productos que hacen lo mismo: AWS IoT Greengrass, Azure IoT Edge o el kubelet de Kubernetes. El patrón técnico es un **bucle de reconciliación**: compara cómo debería estar el servidor con cómo está, y corrige las diferencias.

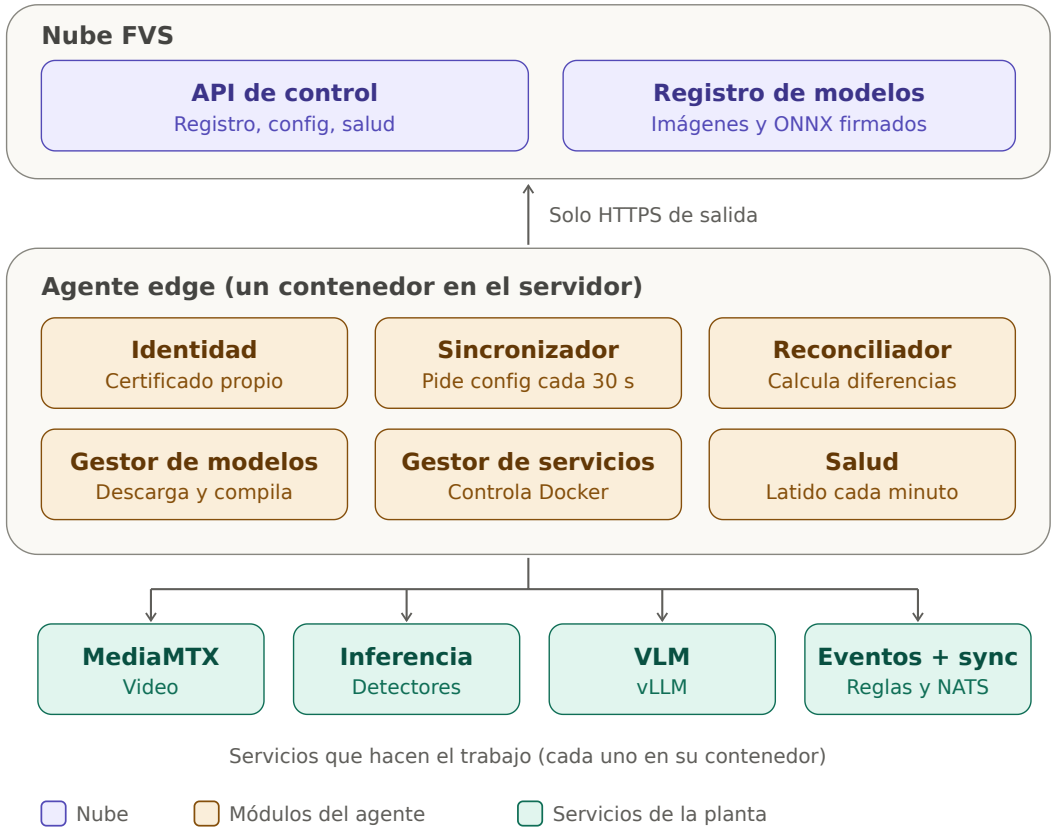


Figura 4. Arquitectura interna del agente edge.

Sus seis módulos

Módulo	Qué hace
Identidad	Guarda el certificado único del servidor y lo renueva antes de que caduque.
Sincronizador	Pide la configuración cada 30 s. Sin internet, trabaja con la última guardada en disco.

Reconciliador	Compara la configuración nueva con la actual y arma el plan de cambios. Aplica solo diferencias.
Gestor de modelos	Descarga ONNX, verifica firma, compila a TensorRT y conserva la versión anterior para regresar.
Gestor de servicios	Controla los contenedores vía Docker: arranca, actualiza o reinicia servicios y les entrega su configuración.
Salud	Cada minuto reporta GPU, memoria, temperatura, disco, cámaras conectadas y fps reales.

¿En qué lenguaje?

Python es lo más práctico para el MVP, empaquetado como contenedor Docker. En producción muchos agentes se escriben en Go (un solo ejecutable, poca memoria, fácil de distribuir); se puede reescribir después sin cambiar la API de la nube. El corazón del agente es este ciclo:

```
while True:
    deseado = api.obtener_config()                    # pregunta a la nube con su certificado
    actual = estado_local.leer()                      # lo aplicado, guardado en disco
    if deseado.version != actual.version:
        plan = calcular_diferencias(deseado, actual) # qué agregar, quitar o cambiar
        try:
            for paso in plan:                         # descargar modelo, agregar cámara...
                paso.aplicar()
            estado_local.guardar(deseado)
            api.reportar(version=deseado.version, ok=True)
        except Exception as e:
            revertir_a(actual)                        # regresa a la última versión que funcionaba
            api.reportar(version=deseado.version, ok=False, error=str(e))
    api.latido(salud.medir())                         # GPU, disco, cámaras, fps
    time.sleep(30)
```

11. Instalación y enrolamiento

Instalación con un solo comando

Se descarga desde FVS y se ejecuta con bash en el servidor de la GPU, igual que Docker, Tailscale o K3s:

```
curl -fsSL https://fvs.tuempresa.com/install.sh | sudo bash -s -- --token FVS-7K2Q-9XM4
```

1. Verifica el sistema: Ubuntu, GPU NVIDIA (`nvidia-smi`), espacio en disco y salida a internet.
2. Instala lo que falte: Docker y NVIDIA Container Toolkit.
3. Descarga la imagen del agente y la arranca con reinicio automático.
4. El agente usa el token para registrarse en la nube.
5. A partir de ahí, el agente descarga y arranca solo todo lo demás.

Alternativas para empresas estrictas: el mismo instalador como paquete `.deb` firmado desde un repositorio apt, o una imagen de instalación completa (sistema operativo + Docker + agente) al

estilo appliance cuando haya muchas plantas.

Enrolamiento: cómo se vincula con la nube

1. En el portal, un administrador elige “Agregar servidor” en la planta y obtiene un **token de un solo uso** que caduca en unas horas.
2. En la planta, el técnico ejecuta el instalador con ese token.
3. El agente genera su propio par de llaves; la privada nunca sale del servidor.
4. Envía a la nube el token, su llave pública y los datos de su hardware (GPU, memoria, disco).
5. La nube valida y quema el token, registra el servidor y devuelve un **certificado firmado** (por ejemplo, con Certificate Authority Service de GCP).
6. El portal muestra el servidor como “Conectado”. Desde ahí, el agente se autentica siempre con su certificado.

Seguridad

- **Solo conexiones de salida (HTTPS):** en la planta no se abre ningún puerto, lo que facilita la aprobación de TI.
- **Ningún servidor guarda llaves de GCP:** para descargar un modelo, la API de control entrega una URL firmada de corta duración. Si roban un servidor, se revoca su certificado desde el portal.
- **Sin comandos libres:** el agente solo ejecuta acciones de una lista cerrada (aplicar configuración, reiniciar un servicio, enviar logs). Nunca abre una terminal remota.

> No usar Google Cloud IoT Core, que aparece en tutoriales viejos: Google lo descontinuó en 2023. Una API de control propia en Cloud Run es más simple y no depende de ese servicio.

12. Qué instala el agente

El instalador solo pone tres cosas: Docker, NVIDIA Container Toolkit y el agente. **Todo lo demás lo instala el agente**, pero no a la manera tradicional: cada servicio viene como **imagen de contenedor** con todo lo que necesita. El agente descarga las imágenes y las arranca. Cambiar de versión es cambiar de imagen, y regresar es igual de fácil. En la práctica, el agente administra algo equivalente a un `docker-compose.yml` de la planta, que nadie escribe a mano.

Orden	Qué hace el agente	Detalle
1	Pide su primera configuración	Incluye la lista de servicios y versiones: MediaMTX, inferencia, VLM, motor de eventos, NATS, sincronización.
2	Descarga imágenes	Desde Artifact Registry.
3	Descarga modelos	ONNX de detectores (MB) y pesos del VLM (varios GB).
4	Arranca servicios en orden	Primero NATS y MediaMTX; luego inferencia y VLM; al final eventos y sincronización.
5	Entrega configuración	Cámaras a MediaMTX, modelos a inferencia, reglas al motor de eventos.
6	Reporta	El portal muestra la planta lista.

El VLM es el caso especial

- Guardar los modelos en una carpeta del disco fuera de los contenedores, para no volver a descargarlos al actualizar o reiniciar.
- Reanudar la descarga si se corta, en lugar de empezar de cero.
- Arrancar detectores y alertas sin esperar al VLM: la planta opera mientras el VLM termina de bajar.

Actualización del propio agente

La versión del agente va en la configuración deseada. Como un programa no puede reemplazarse a sí mismo mientras corre, un proceso mínimo de apoyo (actualizador) descarga la nueva imagen, la arranca, verifica que se reporte sano y, si falla, regresa a la anterior. Primero en una planta piloto, luego en el resto.

13. ¿Hace falta K3s?

No es obligatorio. Depende del tamaño:

Opción	Cuándo usarla	Ventaja	Desventaja
Docker Compose + agente edge	1 servidor por planta, hasta unas 10 plantas	Lo más simple de instalar y operar	La lógica de actualización se programa en el agente
K3s + GitOps (Fleet o Argo CD)	Varios servidores por planta o decenas de plantas	Autorreparación y despliegues estandarizados en toda la flota	Más piezas que mantener
Plataforma edge administrada del proveedor de nube	Clientes que ya la usan	Menos operación propia	Dependencia del proveedor y costo

Recomendación: empezar con Docker Compose y el agente propio. Un solo servidor por planta no necesita Kubernetes. Si después se migra a K3s, el agente se adelgaza (la herramienta GitOps reconcilia contenedores), pero las partes propias del dominio (compilar TensorRT, configurar MediaMTX) se conservan.

14. Cómo mantenerlo fácil y sencillo

1. **Un appliance por planta:** instalador de un comando o imagen completa; el técnico ingresa un token y la planta aparece en el portal.
2. **Estado deseado, no órdenes:** el portal describe cómo debe estar la planta y el agente se encarga de que así sea. Si un servidor se reinstala, se reconstruye solo.
3. **Solo conexiones de salida:** sin puertos abiertos en la planta.
4. **Modelos y software empaquetados igual:** artefactos firmados en Artifact Registry y Cloud Storage, con versionado y verificación de firma.
5. **Funciona sin internet:** la planta sigue detectando, describiendo y alertando; sincroniza al volver la conexión.

6. **Despliegue gradual con rollback automático:** toda versión nueva entra primero en una cámara o planta piloto.

15. Qué hay que construir

Lado	Componentes
Nube	**API de control** en Cloud Run con cuatro funciones: enrolar servidores, entregar la configuración de cada planta, recibir latidos y reportes, y generar URLs firmadas para descargar modelos. AlloyDB para servidores, plantas, versiones de configuración y eventos. Pub/Sub para la entrada de eventos.
Planta	**Agente** en Python dentro de un contenedor, más el **script de instalación**. Servicios en contenedores: MediaMTX, inferencia, VLM, motor de eventos, NATS y sincronización.

Simplificaciones para el MVP

- Sin certificados: un token fijo en el `.env`.
- Sin actualizador del agente ni TensorRT (ONNX directo).
- Catálogo como un bucket de GCS con modelos ONNX y un YAML por modelo como ficha.
- Una planta piloto con Docker Compose.

> Lo importante es construir desde el principio el ciclo **pedir configuración → comparar → aplicar diferencias → reportar**, porque todo lo demás (portal, aprobación, canary, analítica global) se agrega encima de ese ciclo cuando se sume la segunda o tercera planta.